# BusinessFlow

Business Operations Management SaaS

| | |
|---|---|
| **Project goal** | Build a portfolio-grade full-stack SaaS app to demonstrate frontend, backend, database, security, testing, and deployment skills. |
| **Frontend** | Next.js, React, TypeScript, Tailwind CSS |
| **Backend** | Python, FastAPI, Pydantic |
| **Database** | PostgreSQL, SQLAlchemy 2.0, Alembic |
| **Target roles** | Full Stack Developer / Product Engineer (React + Python) |

**Use this document as the scope of work. Build the core modules first, then add advanced features only after the fundamentals are stable.**

## What BusinessFlow does

BusinessFlow helps a company manage projects, tasks, people, deadlines, collaboration, and operational reporting in one web application. Multiple organizations can use the platform while their private data stays isolated.

### Product success criteria

- A user can sign in and access only the organization and data they are permitted to see.
- A manager can create projects, assign tasks, track deadlines, and review progress.
- A member can update assigned work and collaborate through comments.
- A recruiter can inspect the deployed app, API documentation, tests, and project README.

# 1. Complete Module List

These are the planned product modules. Treat MVP modules as required; later-phase modules are optional enhancements once the core workflow works end to end.

### 01. Authentication & Account

Signup, login/logout, password hashing, current-user profile, password reset (optional), secure session/token handling.

### 02. Organization / Workspace

Create a company workspace, edit workspace details, switch organizations, and keep all workspace data separate.

### 03. Team & Member Management

View members, add/invite members, assign roles, deactivate membership, and show invitation status.

### 04. Roles & Permissions (RBAC)

Admin, Manager, Member roles; define who can manage settings, projects, tasks, users, and reports.

### 05. Dashboard

KPI cards for total/active projects, open/overdue tasks, completion rate, recent activity, and upcoming deadlines.

### 06. Project Management

Create, view, edit, archive projects; project owner, description, status, start/end dates, team members, and progress.

### 07. Task Management

Create/edit/delete tasks, assign a member, priority, status, due date, description, labels, and subtasks (later phase).

### 08. Kanban Board & Task Views

Move tasks across To Do / In Progress / Review / Done; list view, sorting, filters, search, and pagination.

### 09. Comments & Collaboration

Task comments, mention-ready design (optional), and a timeline of updates.

### 10. Notifications

In-app notifications for assignments, comments, and approaching deadlines; email notifications can be a later phase.

### 11. Search, Filters & Pagination

Search projects/tasks, filter by status, priority, assignee, date range, and paginate server-side.

### 12. Reports & Analytics

Project completion, overdue tasks, workload by member, task trends, and date-range filters.

### 13. CSV Import / Export

Import tasks from CSV with field validation and row-level errors; export filtered task/project data.

### 14. Activity Log / Audit Trail

Record important actions such as project creation, task status changes, role changes, and member removal.

### 15. Settings

Profile settings, workspace settings, default preferences, and membership/role administration.

### 16. AI Business Reports — Future

Ask questions about authorized workspace data or summarize delayed projects. Enforce permissions before any AI retrieval.

# 2. Core User Workflows

| Workflow | Steps |
|---|---|
| New workspace | Sign up → create workspace → become Admin → add team members → create first project. |
| Project delivery | Create project → add tasks → assign owners and deadlines → update status → review progress on dashboard. |
| Team collaboration | Open task → add comment → receive notification → review activity history. |
| Reporting | Choose workspace/date range → view project/task metrics → export CSV. |

# 3. Technology Stack & Architecture

| Layer | Choice | Purpose |
|---|---|---|
| Frontend | Next.js + TypeScript + Tailwind | Pages, forms, dashboard, Kanban, responsive UI. |
| Data fetching | TanStack Query (recommended) | API state, loading/error states, cache invalidation. |
| Backend | FastAPI + Pydantic | REST APIs, validation, auth dependencies, OpenAPI docs. |
| ORM / migrations | SQLAlchemy 2.0 + Alembic | Database access and versioned schema changes. |
| Database | PostgreSQL | Relations, constraints, indexes, transactions. |
| Testing | Pytest + frontend tests | API logic, permissions, critical UI workflows. |
| Delivery | Docker + CI/CD | Repeatable local setup and deployment pipeline. |

### Request flow

**Next.js UI → FastAPI router → authentication & permission checks → service/business logic → SQLAlchemy → PostgreSQL → validated response → UI.**

### Suggested backend folders

```
backend/
app/
api/ # endpoint routers
core/ # config and security
db/ # database session and base
models/ # SQLAlchemy models
schemas/ # Pydantic schemas
services/ # business logic
main.py
tests/
alembic/
requirements.txt
.env.example
```

# 4. Initial Database Design

Start with these tables; add optional tables as the corresponding modules are implemented.

| Table | Important fields / relationships |
|---|---|
| users | id, email, password_hash, full_name, created_at |
| organizations | id, name, created_at |
| memberships | id, organization_id, user_id, role, status; links users to organizations |
| projects | id, organization_id, name, description, status, owner_id, dates |
| tasks | id, organization_id, project_id, title, status, priority, assignee_id, due_date |
| comments | id, task_id, author_id, body, created_at |
| notifications | id, user_id, type, message, read_at, created_at |
| audit_logs | id, organization_id, actor_id, action, entity_type, entity_id, created_at |

### Data rules to plan for

- Use primary keys, foreign keys, unique constraints, and timestamps consistently.
- Scope every organization-owned record to organization_id and verify the current user's membership.
- Use migrations for schema changes; do not rely on manually editing production tables.
- Use transactions when one operation must update multiple related records.

# 5. Suggested API Groups

| Area | Example endpoints |
|---|---|
| Auth | POST /auth/register, POST /auth/login, GET /auth/me, POST /auth/logout |
| Organizations | POST /organizations, GET /organizations, GET/PATCH /organizations/{id} |
| Members | GET /organizations/{id}/members, POST /organizations/{id}/members, PATCH /members/{id} |
| Projects | GET/POST /projects, GET/PATCH/DELETE /projects/{id} |
| Tasks | GET/POST /tasks, GET/PATCH/DELETE /tasks/{id}, PATCH /tasks/{id}/status |
| Comments | GET/POST /tasks/{id}/comments |
| Dashboard/reports | GET /dashboard/summary, GET /reports/projects, GET /reports/tasks |
| Notifications | GET /notifications, PATCH /notifications/{id}/read |

# 6. Build Roadmap (Recommended Order)

### Phase 1 — Foundation

Create repository, virtual environment, FastAPI app, environment settings, health endpoint, and PostgreSQL connection.

### Phase 2 — Database

Set up SQLAlchemy, Alembic, and initial users/organizations/memberships/projects/tasks tables.

### Phase 3 — Authentication & workspace

Implement login, current-user handling, workspace creation, membership, and role checks.

### Phase 4 — Core business modules

Complete Projects, Tasks, Kanban/list views, comments, filters, sorting, and pagination.

### Phase 5 — Frontend integration

Build login, dashboard, projects, task board, task forms, members, and settings pages.

### Phase 6 — Operational modules

Add notifications, audit logs, CSV import/export, and analytics.

### Phase 7 — Quality & security

Write tests, test cross-organization access, handle errors, validate inputs, and review secrets/configuration.

### Phase 8 — Deployment

Dockerize, configure CI/CD, deploy app and database, add demo data, screenshots, and a clear README.

### Phase 9 — Optional AI

Add AI summaries and natural-language reporting only after authorization and reporting data are reliable.

# 7. MVP vs Later Enhancements

| MVP — must build first | Later — after MVP is stable |
|---|---|
| Auth, organization/workspace, members/roles, dashboard, projects, tasks, Kanban/list views, comments, filters, pagination, basic tests. | Email invitations, advanced notifications, CSV bulk import, richer analytics, audit log UI, AI reports, subtasks, advanced search. |

# 8. Quality, Security & Definition of Done

- Passwords are securely hashed and secrets are stored in environment variables, never committed to Git.
- Every protected endpoint checks identity, role, and organization membership on the server.
- Tests verify both allowed access and denied cross-organization access.
- The database can be recreated using migrations; the README explains setup and test commands.
- A deployed demo or reproducible local setup is available, with screenshots and API documentation.

# 9. Resume Project Summary

**BusinessFlow — Multi-Tenant Business Operations SaaS**
**Built a full-stack SaaS platform using Next.js, TypeScript, FastAPI, and PostgreSQL. Implemented organization-based access control, project and task workflows, REST APIs, relational database models, analytics, automated tests, and containerized deployment.**

Only include features and results you have actually implemented and verified. Add real metrics only after measuring them.